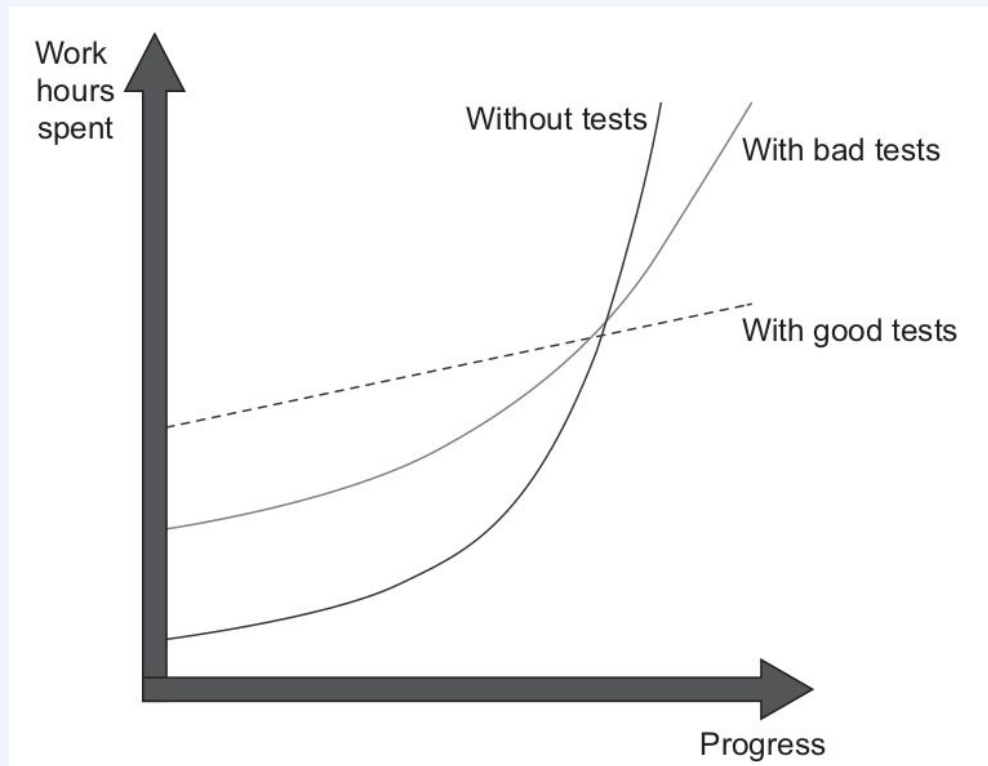


Тестирование

Для чего нужно писать тесты?

Тесты - это также код программы, умеющий своим назначением верификацию корректности работы программы.

Соотношение между функциональным кодом программы и тестами обычно 1:1 - 1:3, иногда до 1:10.



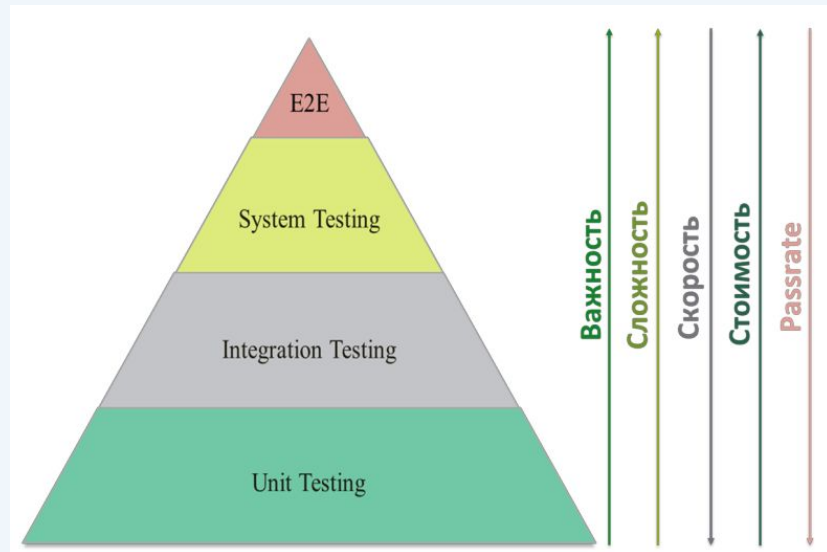
Пирамида тестирования

Уровни тестирования:

Unit/component/program/module testing - тестируется минимально-атомарный модуль программы, чаще всего это одна функция или метод. Таких тестов должно быть больше всего.

Integration testing - несколько модулей программы тестируются вместе.

System testing - вся программа тестируется полностью.



Unit tests

- verifies a small piece of code
- does it quickly
- does it in an isolated manner

Good unit tests:

- protection against regressions
- resistance to refactoring
- fast feedback
- maintainability

Types:

- Black-box - without knowing internal structure (use by default)
- White-box - with verification of inner working (use it only high algorithmic complexity)

AAA pattern / Given-When-Than pattern

```
public class CalculatorTest {  
    public void sum_whenOk() {  
        // 1. Arrange / Given  
        int first = 1;  
        int second = 2;  
        int expected = 3;  
        var calculator = new Calculator();  
  
        // 2. Act / When  
        int actual = calculator.sum(first, second);  
  
        // 3. Assert / Than  
        assertEquals(expected, actual);  
    }  
}
```

Naming:

[methodUnderTest]_[scenario]_[expectedResult]

4. Тест должен оставить после себя окружение в том же состоянии, в котором окружение было до выполнения теста.

F.I.R.S.T. принципы

Fast - тесты должны выполняться быстро. Запуск тестов при каждом изменении кода, если они будут долго выполняться, будет отнимать много времени.

Independent - результаты выполнения одного теста не должны быть входными данными для другого. Все тесты должны выполняться в произвольном порядке, поскольку в противном случае при сбое одного теста каскадно может “накрыться” выполнение целой группы тестов.

Repeatable - тесты должны давать одинаковые результаты независимо от среды выполнения (локальный компьютер или билд-сервер). В обратном случае будет дополнительная сложность.

Self-Validating - результатом выполнения теста должно быть булево значение. Тест либо прошел, либо не прошел и должно быть максимально легко понятно по какой причине тест не прошел. Не должно быть необходимости читать логи для того, чтобы определить почему тест не прошел или прошел успешно.

Timely - тесты должны создаваться своевременно. Несвоевременность написания тестов является главной причиной того, что они откладываются на потом, а “потом” никогда не наступает. Рекомендуется писать тесты или параллельно или перед кодом (**TDD**).

JUnit

JUnit Platform — фундаментальная основа для запуска на JVM фреймворков для тестирования. Платформа предоставляет TestEngine API, для разработки фреймворков (для тестирования), которые могут быть запущены на платформе. Кроме этого, в платформе имеется Console Launcher для запуска платформы из командной строки а также для запуска любого JUnit 4 Runner'a на платформе. Уже, кстати, есть плагины для Gradle и Maven.

JUnit Jupiter — сердце JUnit 5. Этот проект предоставляет новые возможности для написания тестов и создания собственных расширений. В проекте реализован специальный TestEngine для запуска тестов на ранее описанной платформе.

JUnit Vintage — поддержка легаси. Определяется TestEngine для запуска тестов ориентированных на JUnit 3 и JUnit 4.

JUnit - Аннотации

@Test обозначает тестовые методы, методы должны быть `public void`. Здесь размещаются сами проверки. У данной аннотации есть два параметра, `expected` — задает ожидаемое исключение и `timeout` — задает время, по истечению которого тест считается провалившимся.

@Before - обозначает методы, которые будут вызваны до исполнения теста, методы должны быть `public void`. Используется для размещения предустановки для теста, например генерации тестовых данных.

@BeforeClass - обозначает методы, которые будут вызваны до создания экземпляра тест-класса, методы должны быть `public static void`. Имеет смысл размещать предустановки для теста в случае, когда класс содержит несколько тестов использующих различные предустановки, либо когда несколько тестов используют одни и те же данные, чтобы не тратить время на их создание для каждого теста.

@After - обозначает методы, которые будут вызваны после выполнения теста, методы должны быть `public void`. Здесь размещаются операции освобождения ресурсов после теста, очистка тестовых данных.

@AfterClass - связана по смыслу с `@BeforeClass`, но выполняет методы после теста,, методы должны быть `public static void`.

@Ignore - если поместить эту аннотацию на класс, то все тесты в этом классе будут отключены.

JUnit - Аннотации

@ParameterizedTest - to execute a single test method multiple times with different parameters.

@ParameterizedTest

@ValueSource(ints = {1, 3, 5, -3, 15, Integer.MAX_VALUE})

```
void isOdd_shouldReturnTrueForOddNumbers(int number) {  
    assertTrue(Numbers.isOdd(number));  
}
```

```
@ValueSource(strings = {"", " "})  
@NullAndEmptySource,  
@EnumSource(Month.class)  
@EnumSource(  
    value = Month.class,  
    names = {"APRIL", "JUNE", "SEPTEMBER"},  
    mode = EnumSource.Mode.EXCLUDE)
```

JUnit - Аннотации

@ParameterizedTest

@MethodSource("provideStringsForIsBlank")

```
void isOdd_ShouldReturnTrueForOddNumbers(int number) {  
    assertTrue(Numbers.isOdd(number));  
}
```

```
private static Stream<Arguments> provideStringsForIsBlank() {  
    return Stream.of(  
        Arguments.of(null, true),  
        Arguments.of("", true),  
        Arguments.of(" ", true),  
        Arguments.of("not blank", false)  
    );  
}
```

@CsvSource(value = {"test:test", "tEst:test", "Java:java"}, delimiter = ':')

@CsvFileSource(resources = "/data.csv", numLinesToSkip = 1)

Mockito

Mockito - фреймворк для работы с заглушками.

Для создания заглушек (или моков) используется `mock(class)`, после его создания выполняется переопределение его поведения, т.е. указывается возвращаемое значение для метода. Если указать более одного возвращаемого значения, они будут возвращены методом последовательно, пока не вернётся последнее, после этого при последующих вызовах будет возвращаться только последнее значение, таким образом, чтобы метод всегда возвращал одно и то же значение, оно указывается один раз.

Mockito

```
import static org.mockito.Mockito.*;

// mock creation
List mockedList = mock(List.class);
// or even simpler with Mockito 4.10.0+
// List mockedList = mock();

// using mock object - it does not throw any "unexpected interaction" exception
mockedList.add("one");
mockedList.clear();

// selective, explicit, highly readable verification
verify(mockedList).add("one");
verify(mockedList).clear();
```

Mockito

`mock()/@Mock`

`when()/given(), thenReturn()/doThrow`

`spy()/@Spy`

`@InjectMocks`

`verify()`

`times(n), never(), atLeastOnce()`

`any()`

`@Captor`

Фреймворки / инструменты

PowerMock

WireMock

TestContainers

Embedded DataBases (H2, embedded Postgres)

Embedded Kafka

SonarQube, Jacoco

Postman, Insomnia



POSTMAN



Insomnia

Mockito

Maven:

```
<dependency>  
  <groupId>org.mockito</groupId>  
  <artifactId>mockito-core</artifactId>  
  <version>4.10.0</version>  
  <scope>test</scope>  
</dependency>
```

Gradle:

```
testCompile "org.mockito:mockito-core:2.7.7"
```

Литература

- <https://habr.com/ru/post/337700/>
- <https://habr.com/ru/post/120101/>
- <https://javadoc.io/doc/org.mockito/mockito-core/latest/org/mockito/Mockito.html>
- <https://www.youtube.com/watch?v=QG8O-Zou0qI&list=PLZqgWWF4O-zhpYUPLjpe2yfg93s1olElm>
- <https://habr.com/ru/post/444982/>